Nederpelt & Geuvers §§1.1–1.2: Functions, informally

William Sollers

Notes, draft 1

Remark (Status of these sections). Everything here is *informal*. Sections 1.1–1.2 motivate the formal definitions that follow: Λ (Def. 1.3.2), substitution (§1.6), and β-reduction (Def. 1.8.1). Each item below points forward to its formal version.

1 Input-output behaviour of functions (§1.1)

Remark (Functions compute). *Source.* N&G §1.1, p. 1 (PDF p. 30), **verify** page.

- Input values are used to calculate output values.
- Variables in the input act as arbitrary (abstract) values.
- Concrete values replace the abstract values in order to compute.

Convention 1.1 (λ-notation)**.** *Source.* N&G §1.1, p. 1.
$\lambda x.\, x^2 + 1$ means $x \mapsto x^2 + 1$: "the function mapping x to $x^2 + 1$".

Remark (What the λ-calculus is). *Source.* N&G §1.1, p. 1.
The λ-*calculus* is a formalisation of the basic aspects of functions, in particular their construction and use. (This is a description, not a definition; correction 1.)

2 The essence of functions (§1.2)

Remark (Two construction principles, one evaluation rule). *Source.* N&G §1.2, **verify** page.
Construction: abstraction and application. Evaluation: β-reduction.

Definition 2.1 (Abstraction, informal)**.** *Source.* N&G §1.2.
From an expression M and a variable x we construct a new expression $\lambda x.\, M$, called the *abstraction of x over M*. Formal version: Def. 1.3.2(3).

Definition 2.2 (Application, informal)**.** *Source.* N&G §1.2.
From expressions M and N we construct the expression MN, called the *application of M to N*. Parentheses are added where necessary. Formal version: Def. 1.3.2(2).

Example 2.3.
- Application of $\lambda x.x$ to $\lambda y.y$ yields $(\lambda x.x)(\lambda y.y)$.
- Application of f to c yields fc, also written $f(c)$.

Remark (Meaningless expressions). The construction principles also allow expressions with no evident meaning, such as xx (self-application). Ignored for now.

Remark (Understanding: a function versus its output). "Square" is $\lambda x.\, x^2$ over $\mathbb{N}$, the map $x \in \mathbb{N} \mapsto x^2 \in \mathbb{N}$. "Square" is not x^2: x^2 is the abstract output value; "square" is the name of the function.

Remark (Piecewise functions). λ-notation is awkward for functions such as $x \mapsto \begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}$.

Verify whether this is N&G's remark or your own (correction 4).

2.1 Evaluation: β-reduction

Definition 2.4 (Substitution, informal)**.** *Source.* N&G §1.2.

$M[x := N]$ is M with N substituted for the *free* occurrences of x (correction 2). Formal version: §1.6.

Definition 2.5 (β-reduction, informal)**.** *Source.* N&G §1.2.

β-reduction is the formalisation of the process of evaluating a function. An expression of the form $(\lambda x.\, M)N$ can be rewritten to $M[x := N]$; this is the *β-reduction* of $(\lambda x.\, M)N$ to $M[x := N]$, written $(\lambda x.\, M)N \to_\beta M[x := N]$. Formal version: Def. 1.8.1.

Example 2.6. One β-step, then evaluate the substitution (correction 3):

(i) $(\lambda y.\, 5)(3) \to_\beta 5[y := 3] \equiv 5$: replace y by 3 in the expression "5".

(ii) $(\lambda x.\, x)(\lambda y.\, y) \to_\beta x[x := \lambda y.y] \equiv \lambda y.\, y$: replace x by $\lambda y.y$ in "x".

(iii) $(\lambda x.\, x^2 + 1)(3) \to_\beta (x^2 + 1)[x := 3] \equiv 3^2 + 1$: replace x by 3 in "$x^2 + 1$".

Remark (Compatibility, informal). *Source.* N&G §1.2, Example 1.2.4, p. 4 (**verify** number).

Reduction is also possible on suitable *parts* of expressions: when $(\lambda x.\, M)N$ is a subexpression, it may be rewritten to $M[x := N]$, provided the rest of the expression is left unchanged. The rules describing how reduction extends from subexpressions to bigger ones are the *compatibility rules*. Formal version: Def. 1.8.1.

Quantified form.

Informally, for all expressions M, N, L and every variable x:

$$M \to_\beta N \implies ML \to_\beta NL, \quad LM \to_\beta LN, \quad \lambda x.M \to_\beta \lambda x.N.$$

To write: Compare with Def. 1.8.1 when you get there: which clauses does it have, and is anything added or dropped?

Example 2.7. By compatibility, $\lambda z.\, ((\lambda x.x)(\lambda y.y)) \to_\beta \lambda z.\, (\lambda y.y)$. The result is the map $z \mapsto (y \mapsto y)$: a function taking any input z and returning the function $y \mapsto y$.

2.2 Functions of several variables

Remark (Currying). *Source.* N&G §1.2, **verify** page.

λ binds exactly one variable. A function of two variables, $f(x, y)$, takes an ordered pair; λ-notation instead takes the arguments one after another. The two-argument function $(x, y) \mapsto M$ becomes

$$\lambda x.\, (\lambda y.\, M),$$

called its *Curried version* (correction 7). The form $\lambda(x, y).M$ is not a λ-term (correction 6).

Remark (Lean). Lean's `fun x y => M` is notation for `fun x => fun y => M`: Lean functions are Curried in exactly this sense.